you decide to make changes in the config repo. How can you make the applications aware of the configuration changes and have them reload it?

In chapter 4, you learned that you could trigger a configuration refresh operation by sending a `POST` request to the `/actuator/refresh` endpoint provided by Spring Boot Actuator. A request to that endpoint results in a `RefreshScopeRefreshedEvent` event inside the application context. All beans marked with `@ConfigurationProperties` or `@RefreshScope` listen to that event and get reloaded when it happens.

You tried the refresh mechanism on Catalog Service, and it worked fine, since it was just one application, and not even replicated. How about in production? Considering the distribution and scale of cloud native applications, sending an HTTP request to all the instances of each application might be a problem. Automation is a crucial part of any cloud native strategy, so we need a way to trigger a `RefreshScopeRefreshedEvent` event in all of them in one shot. There are a few viable solutions. Using Spring Cloud Bus is one of them.

Spring Cloud Bus (https://spring.io/projects/spring-cloud-bus) establishes a convenient communication channel for broadcasting events among all the application instances linked to it. It provides an implementation for AMQP brokers (like RabbitMQ) and Kafka, relying on the Spring Cloud Stream project you learned about in chapter 10.

Any configuration change consists of pushing a commit to the config repo. It would be convenient to set up some automation to make Config Service refresh the configuration when a new commit is pushed to the repository, completely removing the need for manual intervention. Spring Cloud Config provides a Monitor library that makes that possible. It exposes a